

# Local Integrity Checking for Watermarked LLM Outputs via Error-Correcting Codes

Anonymous ACL submission

## Abstract

Watermarking for large language models (LLMs) has emerged as a key mechanism for identifying and attributing machine-generated content. In practical settings, however, LLM-generated content may undergo post-generation edits. While existing watermarking techniques typically focus on robust watermark detection under such edits, local modifications can subtly change the meaning of an LLM-generated response while leaving its overall watermark signal detectable, potentially leading to wrongful attribution of the modified content to the LLM developer. This motivates local integrity checking for watermarked LLM outputs. In this work, we propose an error-correcting-code-based watermarking framework that enables post-generation edit detection by embedding local structural constraints into the generated token sequence. The framework is accompanied by both a watermark detector and a post-generation edit detector. We evaluate our method on Qwen2.5-7B-Instruct under various editing scenarios and demonstrate strong empirical performance in post-generation edit detection.

## 1 Introduction

As large language models (LLMs) become increasingly widespread, the need for techniques to verify content authenticity, ownership, and responsible usage has become increasingly important. Watermarking addresses this need by embedding hidden signals into LLM outputs that enable content tracking, attribution, and verification, helping prevent misuse and ensure accountability (Aaronson, 2023; Dathathri et al., 2024; Chao et al., 2024).

Existing watermarking methods and their detectors are largely designed for document- or segment-level decisions, focusing on whether text is generated by an LLM or a human author (Zhao et al., 2025; Pan et al., 2025; Li et al., 2024). However, it remains largely unexplored whether such methods

can *localize* critical post-generation edits to watermarked LLM outputs. This limitation is particularly important in practice, as even single-token insertions or deletions can substantially alter meaning without affecting the outcome of a watermark detector. As a result, edited content may still be identified as LLM-generated, potentially misattributing responsibility and harming the reputation of the original content creator (Pang et al., 2024; Zhou et al., 2024).

In this work, we tackle this challenge by studying local integrity checking for watermarked LLM outputs through the detection of post-generation token-level edits. At a high level, our approach embeds *code-constrained* local structure into watermarked LLM outputs, enabling the localization of post-generation edits by identifying violations of these local structural constraints. Inspired by error-correcting codes (ECC) that are designed to detect and correct bit insertions/deletions/substitutions (Levenshtein, 1966; Schoeny et al., 2017; Hamming, 1950), we propose to embed ECC strings directly into the text watermark. Our key idea is to impose a feasible codeword space defined by joint VT and Hamming codes, so that local edits can be detected through violations of this structure.

We accompany the watermarking procedure with a corresponding watermark detector and a post-edit detector, jointly enabling watermark verification and fine-grained edit localization. To minimize the impact of ECC watermarking on text quality, we further apply semantic-balanced vocabulary partitioning, soft logit biasing, and adaptive codeword completion to preserve generation quality while maintaining structural guarantees. The main contributions of this paper are summarized as follows:

- We propose a watermarking framework that embeds error-correcting codes into LLM outputs to enable detection of token edits.
- We design a post-edit detection and localization

- method based on identifying violations of the embedded code constraints.
- We demonstrate strong empirical performance across various mixed-editing scenarios.

**Related Work.** Our work builds on the provably robust watermarking framework of Kirchenbauer et al. (2023); Zhao et al. (2024), which perturbs the model’s logit vector using a green-list mechanism. We are further related to the pattern-based watermarking approach in Chen et al. (2025), which targets order-agnostic LLMs. There also exists a watermarking scheme that utilizes pseudorandom error-correcting codes (Christ and Gunn, 2024), but it focuses on robust watermark detection rather than edit localization. Most closely, our work connects to Xie et al. (2025), which studies post-generation edit detection; however, we differ by embedding ECC as structured strings to enable more systematic localization.

There is also a line of work that proposes detectors to flag *long* content, such as AI-generated content detection (Bao et al., 2024; Chakraborty et al., 2023; Gehrmann et al., 2019; Li et al., 2025; Mitchell et al., 2023; Sadasivan et al., 2023). The watermark *agnostic* approach in Kashtan and Kipnis (2024) seeks finer granularity by applying the Higher Criticism metric to detect sparse anomalies. Lei et al. (2025) introduced a Bayesian framework to flag the LLM-generated segments (Cohen, 2022). While their objective is related to ours, they do not consider post-generation edits made to LLM output. Recent work also studies identifying watermarked spans within mixed-source documents. These approaches, such as Zhao et al. (2025) and Pan et al. (2025), are designed to detect long, contiguous watermarked regions and assume that the underlying watermarked text remains largely intact. In contrast, we focus on detecting token-level edits made to watermarked LLM output.

## 2 Problem Setup

Let  $\mathcal{V}$  be the vocabulary set of an LLM. In this work, we focus on autoregressive LLMs (Radford et al., 2019), in which the output at the current token index  $t$  is conditioned on the context of historical tokens  $x_1, x_2, \dots, x_{t-1}$ .

We consider the edit detection and localization task as follows. Let  $x = \{x_t\}_{t=1}^T$  denote an LLM-generated text with an embedded watermark. After generation, the text may undergo

post-generation edits, including insertions, deletions, and substitutions, resulting in an observed edited text  $x' = \{x'_k\}_{k=1}^{T'}$ . Given only the observed text  $x'$ , our goal is to detect and localize the post-generation edits.

This can be formalized via an algorithm that takes text  $\{x'_k\}_{k=1}^{T'}$  as input and outputs a set of suspected *local* edit indexes  $\{I_1, I_2, \dots, I_a\}$ ,  $I_j \in [T']$ . We evaluate the edit detection performance of an algorithm via the following two metrics: *true positive rate* (TPR) and *false alarm rate* (FAR).

- True positive rate: a true edit at position  $t$  is considered correctly detected if  $t$  is in the output of the detection algorithm. The true positive rate is defined as the proportion of true edits that are successfully detected.
- False alarm rate: for a token  $x_t$  that has not been edited, it is considered a false alarm if the algorithm flags the position  $t$ . The false alarm rate is defined as the proportion of such positions that are incorrectly flagged.

In our numerical experiments, we mainly report these metrics at the block level rather than the exact token level: an edited block is treated as a positive instance, and a clean block is treated as a negative instance. This reflects the primary output of our detector, which identifies suspicious watermark blocks and then provides candidate edit locations within flagged blocks, see Figure 1 for an overview.

## 3 Proposed Watermarking and Edit Detection Methods

Our proposed watermarking scheme is based on two types of error-correcting codes (ECC), the VT code and the Hamming code, which we describe in Section 3.1. Then we present the watermarking scheme in Section 3.2, which is visualized in Figure 1. Section 3.3 presents three complementary mechanisms for reducing the generation distortion caused by watermarking. The edit detection algorithm is presented in Section 3.4.

### 3.1 Error-Correcting Code Design

**VT Codes for Insertion/Deletion Detection.** VT codes are classical single insertion–deletion correcting codes defined over binary sequences (Levenshtein, 1966). For a fixed length  $n$  and syndrome parameter  $a \in \{0, 1, \dots, n\}$ , the VT code

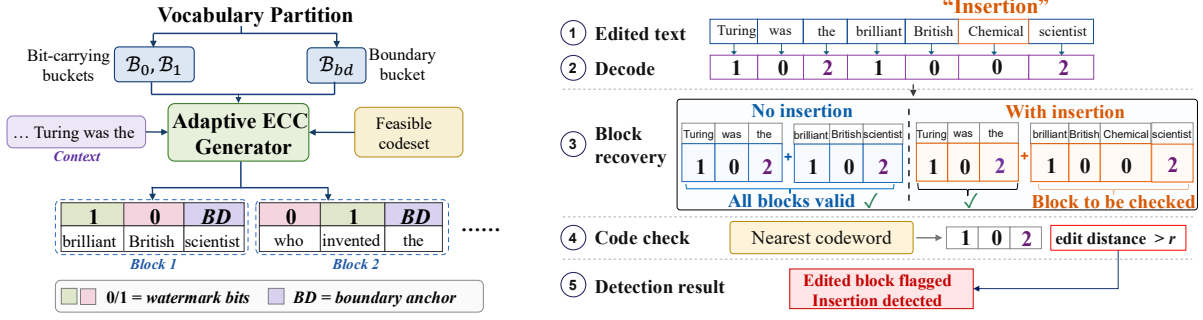


Figure 1: *Left:* Overview of the proposed ECC watermark embedding procedure. The tokenizer vocabulary is partitioned into two buckets  $\mathcal{B}_0, \mathcal{B}_1$  and a disjoint boundary-anchor bucket  $\mathcal{B}_{bd}$ . The feasible codeword set  $\mathcal{C} = \mathcal{V}_a(n) \cap \mathcal{H}$  combines VT and Hamming constraints. During generation, the adaptive ECC generator tracks the current block prefix and restricts or biases the next-token distribution toward feasible bits, while boundary anchors close each structural block. *Right:* Overview of post-generation edit detection and localization using an insertion example. The edited text is first decoded into structural symbols, globally parsed into structural blocks, and then checked against the feasible codeword set  $\mathcal{C}$ . A block is flagged as edited when its minimum edit distance to the feasible set exceeds the tolerance threshold  $r$ .

$\mathcal{V}_a(n) \subset \{0, 1\}^n$  is defined as

$$\mathcal{V}_a(n) = \left\{ x : \sum_{i=1}^n i x_i \equiv a \pmod{n+1} \right\}. \quad (1)$$

A key property of VT codes is that a single insertion or deletion changes the weighted sum constraint in a predictable manner, enabling detection and localization via syndrome decoding. In our watermarking framework, VT constraints are used to enforce block-level structural consistency and enable recovery under insertion and deletion edits.

### Hamming Codes for Substitution Detection.

Hamming codes are linear block codes designed to detect and correct substitution (bit-flip) errors (Hamming, 1950). A binary Hamming code of length  $n$  can be characterized as

$$\mathcal{H} = \left\{ x \in \{0, 1\}^n : Hx^\top = 0 \right\}, \quad (2)$$

where  $H$  is a pre-specified parity-check matrix. If a single bit is flipped, the resulting non-zero syndrome uniquely identifies the error location. In our setting, substitution edits correspond to token replacements that alter the associated watermark bit without changing sequence length. Hamming-type parity constraints therefore provide a complementary detection layer to VT codes.

**Joint Codeword Design.** We combine the VT and Hamming constraints by constructing a feasible codeword space that simultaneously allows detection of insertion-deletion and substitution edits. Let  $\mathcal{V}_a(n)$  denote a VT code with syndrome  $a$ , and let  $\mathcal{H}$  denote a Hamming code of the same

length. In our implementation, given the codewords in Eq. (1) and (2), we define the feasible codeword set as

$$\mathcal{C} = \mathcal{V}_a(n) \cap \mathcal{H}. \quad (3)$$

This intersection represents the set of binary sequences that are simultaneously constrained by the VT condition for insertion-deletion sensitivity and by the Hamming parity condition for substitution sensitivity. The VT syndrome parameter  $a$  is fixed in our experiments, and the Hamming constraint is applied using the standard parity-check matrix over the original coordinate order.

### 3.2 ECC Watermarking

Our watermarking scheme consists of two major components: a mixed-constraint code design with two bit-carrying buckets and a small pool of boundary anchors. At a high level, the language model generates ordinary tokens in its native vocabulary space, while the watermarking layer tracks only a derived symbolic sequence consisting of watermark bits and boundary markers.

**Two Bit-Carrying Buckets.** Recall that  $\mathcal{V}$  denotes the tokenizer vocabulary. After excluding permanently banned tokens and setting aside tokens reserved for boundaries, we partition the remaining usable vocabulary  $\mathcal{V}_{\text{wm}}$  into two buckets,

$$\mathcal{V}_{\text{wm}} = \mathcal{B}_0 \cup \mathcal{B}_1, \quad (4)$$

where  $\mathcal{B}_0$  and  $\mathcal{B}_1$  correspond to watermark bits 0 and 1, respectively. We do not assign punctuation, numbers, or other special lexical categories to separate buckets. Instead, the generator can use such

tokens naturally whenever they remain available. This keeps the construction simple and avoids unnecessary vocabulary fragmentation.

**Boundary Anchors.** To separate consecutive ECC blocks, we reserve a small boundary-anchor pool  $\mathcal{B}_{\text{bd}}$ , disjoint from the two bit-carrying buckets. These anchors are used only to mark block boundaries and are not interpreted as watermark bits. In practice, boundary anchors are selected from tokens that occur relatively infrequently in ordinary text and are excluded from  $\mathcal{B}_0$  and  $\mathcal{B}_1$ , reducing accidental boundary triggers while preserving generation quality.

**Watermarking Process.** As illustrated in Figure 1 (left), during the watermarking process, each closed watermark block consists of  $n$  watermark-bit symbols followed by one boundary anchor. During generation, each emitted token is mapped to either a watermark bit through  $\mathcal{B}_0, \mathcal{B}_1$  or to a boundary anchor through  $\mathcal{B}_{\text{bd}}$ . For a partial block prefix, the generator restricts or biases the next-token distribution toward bit values that keep the prefix compatible with at least one codeword in  $\mathcal{C}$  defined in Eq. (3). Once  $n$  watermark-bit symbols have been generated, the next structural symbol is forced or encouraged to be a boundary anchor, and this closes the current block.

**Watermark Detection.** The presence of such an ECC-based watermark can be detected by checking block-wise ECC adherence. The detector is assumed to have access to the watermarking key, allowing it to recover the vocabulary partition and the set  $\mathcal{C}$ . Specifically, for each block  $j$ , we decode the corresponding binary code string  $\hat{c}^{(j)}$  and check whether it satisfies the VT condition and Hamming condition simultaneously, i.e., whether it belongs to the feasible codeword set  $\mathcal{C}$ . We define the watermark detection statistic as the empirical fraction of blocks that adhere to the ECC constraints, that is,  $\frac{1}{J} \sum_{j=1}^J \mathbb{I}\{\hat{c}^{(j)} \in \mathcal{C}\}$ . Under the null hypothesis of unwatermarked text,  $\mathbb{I}\{\hat{c}^{(j)} \in \mathcal{C}\}$  should approximately have a Bernoulli distribution with mean value much less than one, whereas watermarked text yields  $\mathbb{I}\{\hat{c}^{(j)} \in \mathcal{C}\}$  closer to one, enabling reliable watermark detection.

### 3.3 Distortion-Reduction Mechanisms

A key objective of our framework is to preserve generation quality while allowing mixed-edit detection. Since hard structural constraints can substan-

tially increase perplexity by restricting the token selection space, we therefore design and implement three complementary mechanisms to reduce the distortion to text quality. These three mechanisms are described as follows.

**Soft Watermarking via Logit Biasing.** The hard watermarking enforces bucket constraints by restricting token sampling exclusively to a designated subset. While effective for enforcing structural bits, such hard constraints reduce expressive flexibility and increase perplexity.

Therefore, we also apply soft watermarking by adding a logit bias  $\delta$  toward tokens belonging to the desired bucket, without strictly forbidding alternatives. Formally, for a desired bucket  $\mathcal{B}$ , similar to Aaronson (2023), we modify the next-token logits from the original  $z_t$  to  $\tilde{z}_t$  as

$$\tilde{z}_t(v) = \begin{cases} z_t(v) + \delta, & v \in \mathcal{B}, \\ z_t(v), & v \notin \mathcal{B}. \end{cases}$$

This preserves probabilistic flexibility while statistically encouraging compliance with the structural bit. The bias parameter  $\delta$  governs the detectability–distortion trade-off.

In our implementation, we distinguish hard and soft watermarking by the magnitude of the logit bias. Specifically, we use a large bias ( $\delta = 20$ ) to approximate hard constraints, and a smaller bias ( $\delta = 2$  and  $5$ ) for soft watermarking.

**Semantic-Balanced Bucket Splitting.** Watermark bits are realized through vocabulary partitioning into buckets as in Eq. (4). Naive partitioning can cause certain buckets to lack semantically appropriate tokens, leading to unnatural generations. To mitigate this, we construct semantically balanced buckets. Inspired by the semantic-balanced partitioning strategy of Guo et al. (2024), we construct a static semantic-balanced split over the usable vocabulary  $\mathcal{V}_{\text{wm}}$  by grouping semantically similar tokens and then partitioning each group approximately evenly across the two buckets  $\mathcal{B}_0$  and  $\mathcal{B}_1$ . This preserves expressive capacity within each bucket and reduces the perplexity increase caused by structural constraints.

**Adaptive Codeword Completion.** Both VT and Hamming constraints admit multiple valid codewords per block. Rather than committing to a fixed codeword before generation, we exploit this redundancy adaptively. Recall that  $\mathcal{C} \subseteq \{0, 1\}^n$  in

Eq. (3) denotes the feasible codeword set obtained from the intersection of the VT and Hamming constraints. During generation, we maintain the set of feasible codewords consistent with the current generated prefix. If the current prefix is  $p \in \{0, 1\}^\ell$  with  $\ell < n$ , define

$$\mathcal{C}(p) = \{c \in \mathcal{C} : c_{1:\ell} = p\}.$$

At each step, we first allow the language model to propose the next token under its original distribution. If the corresponding bit keeps the feasible set nonempty, i.e., if the updated prefix  $p'$  satisfies  $\mathcal{C}(p') \neq \emptyset$ , we do not intervene. Otherwise, we restrict or bias the next-token distribution toward bit values that keep the generation on at least one feasible codeword path. In this way, the specific codeword is selected implicitly by the LLM’s original next-token distribution, and constraints are imposed only when necessary. This adaptive completion mechanism reduces forced token selection and helps preserve generation quality while maintaining consistency with the feasible codeword space.

These three mechanisms jointly enlarge the effective feasible region during generation. Soft biasing preserves probabilistic support, semantic-balanced splitting preserves lexical diversity, and adaptive completion exploits code redundancy. The ablation study in Section 4 shows that they can effectively improve perplexity relative to hard watermarking.

### 3.4 Edit Detection

The detection of post-generation edits is performed in two steps, as visualized in Figure 1 (right). First, we decode the observed token sequence to recover the ECC blocks and their associated information bits. We then check whether the recovered bits are consistent with the code space  $\mathcal{C}$ .

**Decoding.** We first map each token to its bucket label, obtaining a structural sequence over  $\{0, 1, 2\}$ , where 0 and 1 denote watermark bits and 2 denotes a boundary anchor. Under mixed post-generation edits, boundary anchors may be deleted or substituted to disrupt local alignment. Therefore, rather than relying on observed boundary symbols alone, we formulate boundary recovery as a global decoding problem that searches for the minimum-cost block decomposition of the entire structural sequence.

Let  $y := \{x'_k\}_{k=1}^{T'}$  denote the observed token sequence. We consider all possible segmentations of  $y$  into candidate blocks, together with assignments

of codewords and boundary states. Each block is associated with: (i) a bit-carrying segment, and (ii) a boundary state indicating whether the boundary is intact, deleted, or substituted.

We define the total decoding cost as

$$\mathcal{L} = \sum_k \left( d_{\text{edit}}(y_k, c_k) + c_{\text{bd}}(k) \right), \quad (5)$$

where  $y_k$  is the observed token subsequence assigned to block  $k$ ,  $c_k \in \mathcal{C}$  is a candidate codeword,  $d_{\text{edit}}$  is the edit distance between the observed subsequence and the codeword, and  $c_{\text{bd}}(k)$  is a boundary penalty defined as

$$c_{\text{bd}}(k) = \begin{cases} 0, & \text{boundary intact,} \\ 1, & \text{boundary deleted or substituted.} \end{cases}$$

Then we recover the blocks by *minimizing* the total decoding cost  $\mathcal{L}$  in Eq. (5) over all possible block segmentations, codeword assignments, and boundary states. We implement this optimization using dynamic programming, which efficiently enumerates candidate segmentations and returns the minimum-cost decomposition. This decoding procedure can thus be viewed as a form of minimum-cost sequence parsing under edit noise. Boundary positions are treated as latent variables inferred through global consistency rather than explicitly detected.

*Remark 1.* We note that block-boundary recovery does not require every anchor to remain intact. Theoretically, under a sparse independent edit model with per-token edit rate  $\epsilon \ll 1$ , the probability that two consecutive boundary anchors are both deleted is on the order of  $\epsilon^2$ , which is negligible for small  $\epsilon$ . Thus, an isolated missing boundary can typically be resolved using adjacent intact anchors and the global block-decomposition decoder in Eq. (5), reducing the risk of cascading alignment errors.

*Remark 2 (Implicit Block Count Determination).* Although the number of blocks is not explicitly specified, the decoding objective implicitly regularizes the segmentation. Since each block is matched with a fixed-length codeword and boundary edits incur additional cost, segmentations with unrealistic block counts (either too many or too few) result in higher total edit distance. As a result, the optimal solution naturally recovers a block count that approximately matches the ground truth.

**Detection.** Given the decoded block decomposition, we analyze the parsed bit-carrying sequence

block by block. For each block  $j = 1, \dots, J$ , let  $\hat{c}^{(j)}$  denote the bit sequence assigned to that block by the decoder. We compare each parsed block symbol  $\hat{c}^{(j)}$  with the feasible codeword set  $\mathcal{C}$  under edit distance. For a nonnegative integer detection threshold parameter  $r$ , block  $j$  is treated as structurally consistent if

$$\min_{c \in \mathcal{C}} d_{\text{edit}}(\hat{c}^{(j)}, c) \leq r.$$

Otherwise, the block is flagged as suspicious and treated as containing a post-generation edit.

Under hard watermarking and the approximate-hard setting, we use  $r = 0$ . For finite soft-bias settings, model outputs may occasionally produce blocks that deviate from the VT-Hamming feasible set even before any post-generation edit is applied. Therefore, to reduce false alarms caused in soft-bias regimes, we use  $r = 2$  for  $\delta = 2$  and  $r = 1$  for  $\delta = 5$  in our experiments.

**Edit Localization.** In addition to block-level detection, the detector can return a candidate set of possible edit locations for each flagged block. Under hard watermarking, where the ECC structure is strictly enforced, these candidates can be derived from the error-correction rules of the underlying codes. More generally, especially under soft watermarking, we obtain the candidate set from the minimum-edit-distance alignments between the parsed block and the codeword set  $\mathcal{C}$ . In Section 4, we report edit-localization coverage as the fraction of true edit events whose ground-truth *token* location is contained in the returned candidate set.

## 4 Numerical Experiments

**Model and Data.** We use Qwen2.5-7B-Instruct as the base language model and generate continuations from a fixed set of Wikipedia-style prompts (Merity et al., 2017). All main experiments use 256 prompts, a target of 18 watermark blocks per continuation, and a maximum generation budget of 160 tokens. Unless otherwise specified, experiments use CUDA inference with 4-bit quantization.

To evaluate edit detection performance, we apply synthetic post-generation edits to the generated text. Specifically, we consider token-level insertions, deletions, and substitutions applied uniformly at random. We test different edit count regimes. In particular, for the single-edit setting, we use a uniform-1-to- $k$  sampling scheme with  $k = 1$ , where each selected block is edited exactly

once. We also consider  $k = 2$  and  $k = 3$ , where each selected block undergoes two or three random edits, respectively. The edit rate parameter controls the fraction of tokens (or blocks) affected by these operations.

**Baselines.** We include the KGW watermarking scheme (Kirchenbauer et al., 2023) as a standard token-level watermarking baseline. Since KGW is not designed for explicit edit localization, we adapt it to our block-wise edit-detection protocol by applying the same synthetic mixed-edit attacks and reporting block-level detection metrics. We further compare against two recent watermark-segment localization methods, Zhao-AOL (Zhao et al., 2025) and WaterSeeker (Pan et al., 2025). Since both methods operate on token-level watermark scores rather than defining standalone watermarking schemes, we instantiate them using the same KGW-style red-green watermark source. We generate KGW-watermarked continuations under the same prompts, length budget, and mixed-edit protocol, and partition them into fixed 8-token blocks to match our ECC block length (with the boundary token). For each edited sequence, token-level KGW watermark-deficit scores are passed to Zhao-AOL or WaterSeeker, and the resulting token/span predictions are mapped to block-level edit predictions by overlap with edited block spans.

**Evaluation Metrics.** We evaluate edit detection performance at both the block level and token level. Block-level detection measures whether a corrupted block is correctly identified, while token-level localization evaluates whether the position of the edit can be accurately covered in the returned candidate set of the detector. We also report generation quality using perplexity.

**Block-level Detection Results.** Table 1 reports block-level post-generation edit detection performance, measured by true positive rate (TPR) and false alarm rate (FAR), under edit budgets  $k = 1, 2, 3$  and logit-bias strengths  $\delta = 2, 5, 20$ , where  $\delta = 20$  represents the approximate-hard regime. Results are averaged over edit rates  $\{0.2, 0.4, 0.6, 0.8\}$ . For ECC, we use distance tolerances 2, 1, and 0 for  $\delta = 2, 5, 20$ , respectively, to account for variation under weaker bias.

The results show a clear TPR-FAR trade-off under finite-bias watermarking. At small bias ( $\delta$ ) values, KGW and AOL often obtain high TPR by over-flagging, with FAR close to 0.89 for KGW

| $k$ | Method          | $\delta = 2$   |                  | $\delta = 5$   |                  | $\delta = 20$  |                  |
|-----|-----------------|----------------|------------------|----------------|------------------|----------------|------------------|
|     |                 | TPR $\uparrow$ | FAR $\downarrow$ | TPR $\uparrow$ | FAR $\downarrow$ | TPR $\uparrow$ | FAR $\downarrow$ |
| 1   | Ours (ECC)      | 0.2692         | 0.1696           | 0.4432         | 0.3595           | 0.9965         | 0.0015           |
|     | KGW             | 0.9413         | 0.8896           | 0.8636         | 0.6284           | 0.7116         | 0.0790           |
|     | KGW+AOL         | 0.7085         | 0.6202           | 0.7437         | 0.5526           | 0.7714         | 0.1341           |
|     | KGW+WaterSeeker | 0.2904         | 0.2156           | 0.2824         | 0.1565           | 0.3355         | 0.0298           |
| 2   | Ours (ECC)      | 0.2930         | 0.1839           | 0.5516         | 0.3560           | 0.9676         | 0.0035           |
|     | KGW             | 0.9541         | 0.8941           | 0.8948         | 0.6369           | 0.7974         | 0.0976           |
|     | KGW+AOL         | 0.7217         | 0.6037           | 0.7560         | 0.5359           | 0.8029         | 0.1627           |
|     | KGW+WaterSeeker | 0.3007         | 0.2047           | 0.3035         | 0.1498           | 0.3479         | 0.0233           |
| 3   | Ours (ECC)      | 0.3241         | 0.1863           | 0.5889         | 0.3564           | 0.9667         | 0.0064           |
|     | KGW             | 0.9587         | 0.8901           | 0.9098         | 0.6468           | 0.8356         | 0.1244           |
|     | KGW+AOL         | 0.7302         | 0.5865           | 0.7629         | 0.5106           | 0.8122         | 0.1786           |
|     | KGW+WaterSeeker | 0.3068         | 0.1885           | 0.3092         | 0.1354           | 0.3549         | 0.0255           |

Table 1: Comparison of ECC and KGW-based baselines under different logit-bias strengths. Results are averaged across edit rates for each edit budget  $k$ . For ECC, tolerance values are 2, 1, and 0 for  $\delta = 2, 5$ , and 20, respectively. The  $\delta = 20$  setting acts as an approximate-hard regime for ECC.

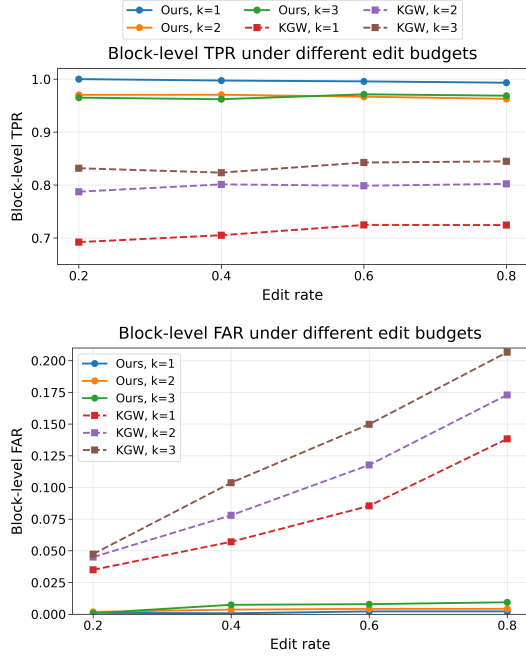


Figure 2: Block-level detection performance under increasing edit rates under  $k = 1, 2, 3$ . Top: TPR. Bottom: FAR. Our method remains robust as  $k$  increases, while KGW degrades significantly.

and above 0.58 for AOL at  $\delta = 2$ . WaterSeeker is more conservative but has low TPR overall. In contrast, our ECC-based method under  $\delta = 2$  and  $\delta = 5$  yields a more controlled TPR–FAR profile: although its TPR is lower than the KGW baseline, its FAR is also substantially lower, avoiding the degenerate “flag almost everything” behavior.

Increasing  $\delta$  strengthens the ECC-induced structural signal and improves edit-detection sensitivity. In the approximate-hard proof-of-concept setting

| $k \setminus$ Edit Rate | 0.2    | 0.4    | 0.6    | 0.8    |
|-------------------------|--------|--------|--------|--------|
| 1                       | 0.9947 | 0.9929 | 0.9932 | 0.9815 |
| 2                       | 0.7151 | 0.7219 | 0.7180 | 0.7042 |
| 3                       | 0.5350 | 0.5402 | 0.5191 | 0.5251 |

Table 2: Edit-location candidate coverage of our method under the approximate-hard ECC setting ( $\delta = 20$ ) with adaptive generation. A localization is counted as correct when the decoder’s candidate edit-location set contains the true edited position.

$\delta = 20$ , ECC achieves TPR consistently above 0.96 while keeping FAR below 0.007 in all cases. By comparison, KGW and AOL obtain lower or comparable recall only with substantially larger FAR, while WaterSeeker keeps FAR low but suffers from low TPR. Figure 2 further visualizes the TPR and FAR as functions of the edit rate under the approximate-hard setting. Overall, these results show that our ECC structure yields more reliable block-level edit detection than baseline watermarking methods and their localization layers once the structural signal is sufficiently stabilized.

**Edit-location Candidate Coverage.** We further evaluate the localization capability of our method under different edit rates and edit budgets. Table 2 reports the edit-location candidate coverage for  $k = 1, 2, 3$  across edit rates from 0.2 to 0.8.

A localization is counted as correct if the true edited position is contained within the candidate edit-location set returned by the decoder. When  $k = 1$ , the method achieves near-perfect localization coverage across all edit rates, consistent

with the single-edit correction capability of VT and Hamming-based decoding. As the edit budget increases to  $k = 2$  and  $k = 3$ , localization performance degrades more noticeably, reflecting the increased ambiguity in recovering multiple edits within a block. Nevertheless, the method retains meaningful localization capability even at higher edit intensities, demonstrating that the proposed decoding framework continues to provide useful edit-localization signals beyond the single-edit regime.

**LLM-guided Sparse Edits.** Beyond random synthetic edits, we further evaluate our method under an LLM-guided editing setting, which is designed to simulate more realistic post-generation editing behavior. We focus on sparse edits in this experiment, where the editor modifies only a small number of local regions. Starting from filtered ECC-generated continuations under the approximate-hard setting, an instruction model proposes sparse edit instructions under both benign motivations, such as grammar polishing and clarity improvement, and malicious motivations, such as claim distortion, stance shift, and source spoofing. The editor does not directly rewrite the entire text. Instead, it outputs structured edit instructions, which are then applied deterministically to the saved ECC structural representation. This gives exact block-level ground truth for evaluation.

Table 3 reports block-level detection performance on the accepted LLM-guided edits. Across 49 accepted edited cases, each containing on average 1.98 edited blocks, our detector achieves TPR 0.9794 and FAR 0.0217. The detector remains effective for both benign and malicious edit intents. In particular, benign edits are detected with TPR 1.0000 at FAR 0.0260, while malicious edits are detected with TPR 0.9600 at FAR 0.0175. These results indicate that the proposed structural watermark remains effective in terms of edit detection under semantically motivated edits, beyond the random edits used in previous experiments.

**Perplexity.** We examine how adaptive generation and logit-bias strength  $\delta$  affect generation quality. Table 4 reports perplexity (PPL) for  $\delta \in \{2, 5, 20\}$ , each with and without adaptive generation, where  $\delta = 20$  is used as an approximate-hard watermarking setting. Adaptive generation consistently reduces perplexity across all bias strengths. For example, at  $\delta = 20$ , adaptive generation reduces PPL from 64.0743 to 23.5530, showing that adaptive codeword realization substantially mitigates

| Edit Type | Cases | TPR $\uparrow$ | FAR $\downarrow$ | Cov. $\uparrow$ |
|-----------|-------|----------------|------------------|-----------------|
| Overall   | 49    | 0.9794         | 0.0217           | 0.3218          |
| Benign    | 24    | 1.0000         | 0.0260           | 0.3224          |
| Malicious | 25    | 0.9600         | 0.0175           | 0.3212          |

Table 3: Block-level detection under sparse LLM-guided edits. Cov. denotes edit-location candidate coverage, i.e., whether the true edit event is included in the decoder’s candidate edit-location set. The LLM editor proposes sparse structured edits, and the edits are applied deterministically to the ECC structural sequence.

|              | $\delta = 2$ | $\delta = 5$ | $\delta = 20$ |
|--------------|--------------|--------------|---------------|
| w/o Adaptive | 13.4063      | 26.9141      | 64.0743       |
| w/ Adaptive  | 10.2773      | 15.6775      | 23.5530       |

Table 4: Perplexity under different soft logit-bias strengths. Results are computed on clean generated continuations without edits. Adaptive generation consistently reduces perplexity across all bias strengths. Larger  $\delta$  provides stronger structural bias but increases generation distortion, with  $\delta = 20$  used as an approximate-hard setting.

the distortion introduced by strong structural bias. As expected, increasing  $\delta$  makes the embedded ECC structure more stable for edit detection, but also increases generation distortion. This trend highlights the trade-off between edit-detection accuracy and generation quality, while confirming that adaptive generation improves fluency even in the approximate-hard setting. Based on this result, all ECC experiments in the paper use adaptive generation as shown in Figure 1 (left).

## Limitations

Our current study focuses on detecting sparse token-level insertions/deletions/substitutions in watermarked LLM outputs. This setting captures an important class of local edits but does not cover longer-span rewrites, which may require more redundant code constructions or layered watermark structures. Our detector also identifies structurally suspicious blocks but does not determine the semantic intent or severity of an edit, such as whether it is benign or malicious. Finally, our empirical evaluation is limited to one open-source model. Since the mechanism operates through token-level vocabulary partitioning and decoding, we expect it to be broadly applicable to other language models, but larger-scale evaluations across additional models, domains, and naturally occurring edits remain important future work.

## References

- Scott Aaronson. 2023. Watermarking of large language models. In *Workshop on Large Language Models and Transformers*. Simons Institute, UC Berkeley.
- Guangsheng Bao, Yanbin Zhao, Zhiyang Teng, Linyi Yang, and Yue Zhang. 2024. Fast-detectGPT: Efficient zero-shot detection of machine-generated text via conditional probability curvature. In *The Twelfth International Conference on Learning Representations*.
- Souradip Chakraborty, Amrit Singh Bedi, Sicheng Zhu, Bang An, Dinesh Manocha, and Furong Huang. 2023. On the possibilities of AI-generated text detection. arXiv preprint arXiv:2304.04736.
- Patrick Chao, Yan Sun, Edgar Dobriban, and Hamed Hassani. 2024. Watermarking language models with error correcting codes. arXiv preprint arXiv:2406.10281.
- Ruibo Chen, Yihan Wu, Yanshuo Chen, Chenxi Liu, Junfeng Guo, and Heng Huang. 2025. A watermark for order-agnostic language models. In *Proceedings of the Thirteenth International Conference on Learning Representations*.
- Miranda Christ and Sam Gunn. 2024. Pseudorandom error-correcting codes. In *Annual International Cryptology Conference*, pages 325–347. Springer.
- Shay Cohen. 2022. *Bayesian analysis in natural language processing*. Springer Nature.
- Sumanth Dathathri, Abigail See, Sumedh Ghaisas, Posen Huang, Rob McAdam, Johannes Welbl, Vandana Bachani, Alex Kaskasoli, Robert Stanforth, Tatiana Matejovicova, and 1 others. 2024. Scalable watermarking for identifying large language model outputs. *Nature*, 634(8035):818–823.
- Sebastian Gehrmann, Hendrik Strobelt, and Alexander Rush. 2019. GLTR: Statistical detection and visualization of generated text. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pages 111–116.
- Yuxuan Guo, Zhiliang Tian, Yiping Song, Tianlun Liu, Liang Ding, and Dongsheng Li. 2024. Context-aware watermark with semantic balanced green-red lists for large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 22633–22646, Miami, Florida, USA. Association for Computational Linguistics.
- Richard W Hamming. 1950. Error detecting and error correcting codes. *The Bell system technical journal*, 29(2):147–160.
- Idan Kashtan and Alon Kipnis. 2024. An Information-Theoretic Approach for Detecting Edits in AI-Generated Text. *Harvard Data Science Review*, (Special Issue 5).
- John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. 2023. A watermark for large language models. In *Proceedings of the International Conference on Machine Learning*, pages 17061–17084.
- Eric Lei, Hsiang Hsu, and Chun-Fu Chen. 2025. Pald: Detection of text partially written by large language models. In *Proceedings of the Thirteenth International Conference on Learning Representations*.
- Vladimir Iosifovich Levenshtein. 1966. Binary coors capable or ‘correcting deletions, insertions, and reversals. In *Soviet Physics-Doklady*, volume 10.
- Xiang Li, Feng Ruan, Huiyuan Wang, Qi Long, and Weijie J Su. 2025. Robust detection of watermarks for large language models under human edits. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, page qkaf056.
- Xingchi Li, Guanxun Li, and Xianyang Zhang. 2024. Segmenting watermarked texts from language models. arXiv preprint arXiv:2410.20670.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer sentinel mixture models. In *Proceeding of the International Conference on Learning Representations*.
- Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D Manning, and Chelsea Finn. 2023. DetectGPT: Zero-shot machine-generated text detection using probability curvature. In *Proceedings of the International Conference on Machine Learning*, pages 24950–24962.
- Leyi Pan, Aiwei Liu, Yijian Lu, Zitian Gao, Yichen Di, Lijie Wen, Irwin King, and Philip S Yu. 2025. Waterseeker: Pioneering efficient detection of watermarked segments in large documents. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 2866–2882.
- Qi Pang, Shengyuan Hu, Wenting Zheng, and Virginia Smith. 2024. No free lunch in LLM watermarking: Trade-offs in watermarking design choices. In *Proceedings of the Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, and 1 others. 2019. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9.
- Vinu Sankar Sadasivan, Aounon Kumar, Sriram Balasubramanian, Wenxiao Wang, and Soheil Feizi. 2023. Can AI-generated text be reliably detected? arXiv preprint arXiv:2303.11156.
- Clayton Schoeny, Antonia Wachter-Zeh, Ryan Gabrys, and Eitan Yaakobi. 2017. Codes correcting a burst of deletions or insertions. *IEEE Transactions on Information Theory*, 63(4):1971–1985.

Liyan Xie, Muhammad Siddeek, Mohamed Seif, Andrea J Goldsmith, and Mengdi Wang. 2025. Detecting post-generation edits to watermarked LLM outputs via combinatorial watermarking. arXiv preprint arXiv:2510.01637.

Xuandong Zhao, Prabhanjan Vijendra Ananth, Lei Li, and Yu-Xiang Wang. 2024. Provable robust watermarking for ai-generated text. In *Proceedings of the Twelfth International Conference on Learning Representations*.

Xuandong Zhao, Chenwen Liao, Yu-Xiang Wang, and Lei Li. 2025. Efficiently identifying watermarked segments in mixed-source texts. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6304–6316.

Tong Zhou, Xuandong Zhao, Xiaolin Xu, and Shaolei Ren. 2024. Bileve: Securing text provenance in large language models against spoofing with bi-level signature. *Advances in Neural Information Processing Systems*, 37:56054–56075.

## A Additional Numerical Results and Details

### A.1 Additional Results

Table 5 reports the true positive rate (TPR) and false alarm rate (FAR) at the block level under varying edit rates in the single-edit setting ( $k = 1$ ), where each selected block receives exactly one edit sampled uniformly from substitution, insertion, and deletion. Our ECC-based method achieves near-perfect TPR with extremely low FAR across all edit rates. In contrast, KGW is substantially less reliable, while AOL improves recall at the cost of much higher FAR and WaterSeeker remains conservative but low-recall.

### A.2 Source Text Generation for Question-Aware Editing

For the downstream question-aware editing study, we construct a pool of source continuations from independent English LFQA prompts. Each prompt is a long-form question, and the generated continuation serves as the watermarked answer to be locally edited. This setup is well matched to malicious-edit evaluation: because the source text answers an explicit question, an editor can produce more realistic spoofing or manipulation attempts by changing how the answer addresses the prompt, for example by altering certainty, attribution, numerical claims, or definitional boundaries. We use Qwen3-8B with chat-template prompting and thinking disabled, together with the ECC soft watermark at logit bias

| Edit Rate | Method | TPR $\uparrow$ | FAR $\downarrow$ |
|-----------|--------|----------------|------------------|
| 0.2       | ECC    | <b>1.0000</b>  | <b>0.0011</b>    |
|           | KGW    | 0.6922         | 0.0350           |
|           | AOL    | 0.7983         | 0.0776           |
|           | WS     | 0.4568         | 0.0232           |
| 0.4       | ECC    | <b>0.9973</b>  | <b>0.0007</b>    |
|           | KGW    | 0.7052         | 0.0571           |
|           | AOL    | 0.7737         | 0.1132           |
|           | WS     | 0.3365         | 0.0284           |
| 0.6       | ECC    | <b>0.9957</b>  | <b>0.0022</b>    |
|           | KGW    | 0.7246         | 0.0854           |
|           | AOL    | 0.7702         | 0.1522           |
|           | WS     | 0.2914         | 0.0279           |
| 0.8       | ECC    | <b>0.9932</b>  | <b>0.0022</b>    |
|           | KGW    | 0.7244         | 0.1383           |
|           | AOL    | 0.7433         | 0.1933           |
|           | WS     | 0.2572         | 0.0400           |

Table 5: Block-level edit detection under the single-edit setting ( $k = 1$ ), where each affected block receives one mixed edit sampled from substitution, insertion, and deletion. AOL denotes KGW+AOL, and WS denotes KGW+WaterSeeker.

$\delta = 5$ . The purpose of using a stronger instruction-following model in this auxiliary study is to obtain more meaningful source answers, so that downstream malicious-edit and risk-classification results are easier to interpret semantically rather than being dominated by poor generation quality.

### A.3 LLM-Guided Editing and Downstream Risk Assessment

This appendix describes the sparse LLM-guided editing experiment in more detail. We start from filtered ECC-generated continuations under the approximate-hard setting. For each selected continuation, the editor model is given the source prompt, the generated suffix, and an indexed edit-unit table. Each edit unit contains an approximate token surface, its associated ECC block ID, bit index, and bucket ID. The editor is also given one of six editing motivations. We group grammar\_polish, clarity\_improvement, and style\_softening as benign intents, and claim\_distortion, stance\_shift, and source\_spoofing as malicious or adversarial intents.

The editor does not directly rewrite the full text. Instead, it outputs a JSON object containing local edit instructions, which specifies a text-unit index or insertion gap, optional original text, new content when applicable, and a short reason. The program validates the JSON, tokenizes the proposed new content, maps the new tokens into watermark buckets, and applies the resulting edit deterministically to the saved ECC structural sequence. This design gives exact block-level ground truth while still al-

| LFQA prompt                                                                       | Watermarked source excerpt                                                                                                                                                                                                                                                                                                                                                                                                            | Example edit focus          |
|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------|
| Can you make electricity from radio waves?                                        | Yes, it is possible to generate a small amount of electricity from radio waves through the use of specialized antennas and rectifiers. This technology, known as a “rectenna,” converts electromagnetic radiation directly into direct current (DC) through a process called rectification. While early experiments demonstrated this capability, practical applications remain limited due to low efficiency and small output power. | Feasibility / magnitude     |
| Do you favor the dark matter hypothesis, or is incomplete gravity more plausible? | The “Dark Matter” hypothesis is currently the leading explanation for observed gravitational effects in galaxies and galaxy clusters that cannot yet be accounted for by visible matter alone however, it remains a theoretical construct since direct detection of dark matter particles has not yet been achieved.                                                                                                                  | Certainty / consensus       |
| What is the difference between embezzlement and theft?                            | Embezzlement and theft are often confused, but they differ in key aspects. Embezzlement involves a person who legally possesses property or funds under their care but misuses them for a personal gain. This typically occurs within a trusted relationship, such as an employee overseeing company funds.                                                                                                                           | Definition boundary         |
| Why are there so many volcanic eruptions recently?                                | There has been no significant increase in global volcanic activity compared to historical averages, though some recent eruptions have received more media attention. This heightened coverage may result from improved communication technologies and greater public awareness of geological phenomena. Volcanic formations are natural processes that occur periodically over geologic time.                                         | Media framing / trend claim |

Table 6: Examples of candidate source continuations for the question-aware LLM-guided editing study. The prompts make malicious or spoofing-oriented edits more natural because the editor can target how the answer responds to the question. Excerpts preserve the generated token strings; only whitespace is normalized for readability.

lowing the edit choices to be guided by semantic motivations. We enforce sparsity by requiring the accepted edits to affect at most three blocks.

Table 7 reports the number of accepted cases by intent and detection performance for each category. The ECC detector achieves strong block-level detection across both benign and malicious edit types. All benign edit cases are detected at the block level in this run, while malicious edit cases also retain high TPR. Candidate coverage is lower than block-level TPR, indicating that identifying the edited block is easier than enumerating the precise edit location within the block.

| Category            | Intent    | Cases | TPR $\uparrow$ | FAR $\downarrow$ | Cov. $\uparrow$ |
|---------------------|-----------|-------|----------------|------------------|-----------------|
| clarity_improvement | Benign    | 8     | 1.00           | 0.02             | 0.38            |
| grammar_polish      | Benign    | 12    | 1.00           | 0.01             | 0.40            |
| style_softening     | Benign    | 4     | 1.00           | 0.08             | 0.16            |
| claim_distortion    | Malicious | 7     | 1.00           | 0.02             | 0.37            |
| source_spoofing     | Malicious | 9     | 0.89           | 0.01             | 0.25            |
| stance_shift        | Malicious | 9     | 1.00           | 0.03             | 0.35            |

Table 7: Block-level detection results under sparse LLM-guided edits by category. Cov. denotes edit-location candidate coverage.